

Lab 5: Classification Techniques

Objective

To understand and implement different classification algorithms — Decision Tree, Naive Bayes, and Logistic Regression — using the `scikit-learn` library, and evaluate their performance with accuracy and confusion matrix.

Theory

Classification is a supervised learning task that assigns input data into predefined categories (classes) based on patterns learned from labeled training data.

Common Classification Algorithms

- **Decision Tree:** A tree-like model that splits data based on attribute values to maximize class purity (measured by Gini or Entropy).
- **Naive Bayes:** A probabilistic model based on Bayes' theorem, assuming feature independence.
- **Logistic Regression:** A linear model that predicts probabilities for categorical outcomes using the sigmoid function.

Evaluation Metrics:

- **Accuracy:** Fraction of correct predictions.
- **Confusion Matrix:** Table showing correct and incorrect classifications.
- **Precision, Recall, F1-Score:** Derived from the confusion matrix to measure class-wise performance.

Major Functions Used

- `train_test_split()`: Splits data into training and testing sets.
- `DecisionTreeClassifier`, `GaussianNB`, `LogisticRegression`: Classifier classes.
- `confusion_matrix()`, `classification_report()`: Performance evaluation.

Procedure

1. Import Required Libraries

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.naive_bayes import GaussianNB
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report
```

2. Load Dataset and Prepare Data

We'll use the popular **Iris dataset** available in `sklearn.datasets`.

```
1 from sklearn.datasets import load_iris
2
3 iris = load_iris()
4 X = iris.data
5 y = iris.target
6
7 # Split data into 80% training, 20% testing
8 X_train, X_test, y_train, y_test = train_test_split(X, y,
  test_size=0.2, random_state=42)
```

Explanation: Splitting ensures the model generalizes well and doesn't simply memorize training data.

3. Decision Tree Classifier

```
1 # Initialize and train model
2 dt = DecisionTreeClassifier(criterion='entropy', max_depth=3,
  random_state=0)
3 dt.fit(X_train, y_train)
4
5 # Predict and evaluate
6 y_pred_dt = dt.predict(X_test)
7 print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
8 print(confusion_matrix(y_test, y_pred_dt))
```

Observation: Decision Trees are easy to interpret but prone to overfitting if not pruned or limited in depth.

4. Naive Bayes Classifier

```
1 nb = GaussianNB()
2 nb.fit(X_train, y_train)
3
4 y_pred_nb = nb.predict(X_test)
```

```
5 print("Naive Bayes Accuracy:", accuracy_score(y_test, y_pred_nb))
6 print(confusion_matrix(y_test, y_pred_nb))
```

Observation: Naive Bayes works well for high-dimensional data and text classification, despite its strong independence assumptions.

5. Logistic Regression

```
1 lr = LogisticRegression(max_iter=200)
2 lr.fit(X_train, y_train)
3
4 y_pred_lr = lr.predict(X_test)
5 print("Logistic Regression Accuracy:", accuracy_score(y_test,
6               y_pred_lr))
6 print(confusion_matrix(y_test, y_pred_lr))
```

Observation: Logistic Regression models linear decision boundaries and is effective for linearly separable classes.

6. Compare Classifier Performance

```
1 # Summary table
2 models = ['Decision Tree', 'Naive Bayes', 'Logistic Regression']
3 accuracies = [
4     accuracy_score(y_test, y_pred_dt),
5     accuracy_score(y_test, y_pred_nb),
6     accuracy_score(y_test, y_pred_lr)
7 ]
8
9 for m, acc in zip(models, accuracies):
10     print(f"{m:20s} Accuracy: {acc*100:.2f}%")
```

Example Output:

Decision Tree	Accuracy: 96.67%
Naive Bayes	Accuracy: 96.67%
Logistic Regression	Accuracy: 100.00%

7. Classification Report and Visualization

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Choose one model (e.g., Decision Tree)
5 cm = confusion_matrix(y_test, y_pred_dt)
6 sns.heatmap(cm, annot=True, cmap='Blues', fmt='g')
7 plt.xlabel('Predicted Label')
8 plt.ylabel('True Label')
9 plt.title('Confusion Matrix - Decision Tree')
10 plt.show()
11
```

```
12 print("Classification Report:\n")
13 print(classification_report(y_test, y_pred_dt,
    target_names=iris.target_names))
```

Explanation: A heatmap visually represents true vs predicted classes; the classification report gives detailed precision and recall.

8. Full Example Script

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.naive_bayes import GaussianNB
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
7
8 iris = load_iris()
9 X_train, X_test, y_train, y_test = train_test_split(iris.data,
    iris.target,
    test_size=0.2, random_state=42)
10
11 models = {
12     'Decision Tree': DecisionTreeClassifier(max_depth=3),
13     'Naive Bayes': GaussianNB(),
14     'Logistic Regression': LogisticRegression(max_iter=200)
15 }
16
17 for name, model in models.items():
18     model.fit(X_train, y_train)
19     preds = model.predict(X_test)
20     print(f"\n{name} Accuracy: {accuracy_score(y_test,
    preds)*100:.2f}%")
21     print(confusion_matrix(y_test, preds))
```

Listing 4: Classification Techniques using Scikit-learn

Discussion

Each classification algorithm has strengths and weaknesses:

- Decision Trees provide interpretability but risk overfitting.
- Naive Bayes is fast and simple, ideal for high-dimensional data.
- Logistic Regression offers probabilistic outputs and works well for linear relationships.

Choosing the best model depends on the dataset and problem type.

Exercises

1. Apply these classifiers to a different dataset (e.g., Titanic or Wine dataset).
2. Vary the test size (e.g., 0.3, 0.4) and observe changes in accuracy.
3. Use `max_depth` and `criterion` parameters in Decision Tree and note the effect.
4. Compare accuracy, precision, recall, and F1-score among the three models.
5. Visualize the Decision Tree using `plot_tree()` from `sklearn.tree`.

Outcome

Students will:

- Learn to implement multiple classification models using `scikit-learn`.
- Understand how to evaluate and compare model performance.
- Develop the ability to select the most appropriate classification technique for a given dataset.

